

PS 26045 - IP-SAKTI Sahayak

COMPLETE RESEARCH MASTER REFERENCE

Whole-project engineering and production reference

Multilingual RAG-based IP / Regulatory Assistant for Ayurveda

Scope: research, domain, regulation, corpus, document intelligence, multilingual AI, RAG, classification, evidence, safety, agents, graph, architecture, backend, frontend, persistence, evaluation, security, MLOps, deployment, observability, hardware/cost, competitors, novelty, roadmap, and final architecture audit.

Development hardware: NVIDIA RTX 3050 | 4 GB VRAM | 16 GB RAM | 512 GB SSD

Source discipline: [OFFICIAL PS] | [OFFICIAL SOURCE] | [EXTERNAL RESEARCH] | [ENGINEERING RECOMMENDATION] | [OUR ENHANCEMENT] | [ASSUMPTION] | [DEFERRED]

This document consolidates the available validated project research artifacts into one reference package. Where a later audit corrected an earlier recommendation, the later/corrected position is preferred. Conditional or deferred components are not mandatory merely because they appear in the research.

How to Use This Reference

1. Treat the source discipline labels as mandatory. 2. Implement one phase at a time. 3. Do not bypass acceptance gates. 4. Re-verify current government/regulatory sources before legal production release. 5. Benchmark conditional components such as reranking, agents and graph before locking them.

Primary Included Artifacts

Artifact	Purpose
Deep Technical Research & Roadmap	Detailed architecture, technical choices, repository layout, roadmap and hardware audit.
28-Part Research Compendium	28-part map plus consolidated reference content and final architecture decisions.

Final engineering principle

The project moat is not the LLM. It is authoritative corpus provenance + regulatory classification + jurisdiction isolation + evidence-grounded retrieval + citation verification + safe abstention.

PS 26045 — IP-SAKTI Sahayak

Deep Technical Research, Engineering Roadmap & Production Plan

Multilingual RAG-based IP/Regulatory Assistant for Ayurveda

Target development hardware

Component	Constraint	Engineering implication
GPU	NVIDIA RTX 3050 / 4 GB VRAM	Do not make a large local LLM the primary dependency; reserve GPU for one local generation model when needed.
RAM	16 GB	Keep the service stack lightweight; run embeddings/reranking on CPU where practical.
Storage	512 GB SSD	Use a curated authoritative corpus and avoid mass scraping; keep raw/normalized/indexed data separated.
Target	SIH → credible production path	Design interfaces so storage, model and deployment tiers can change without rewriting the core pipeline.

Core engineering principle: The project is not a generic Ayurveda chatbot. It is a classification-first, jurisdiction-aware, evidence-grounded regulatory decision-support system. The LLM is only one component.

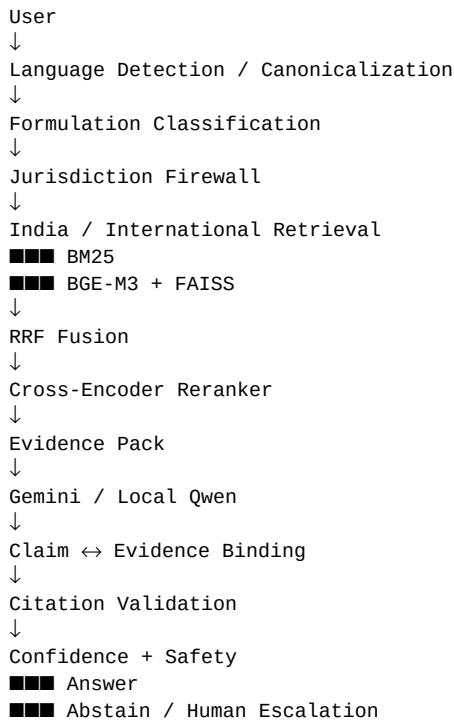
Prepared as a build-before-code research blueprint. Source-derived requirements and architecture are separated from engineering recommendations and future enhancements.

1. Executive Engineering Verdict

The correct starting point is **corpus and retrieval validation**, not UI, agents, voice, a knowledge graph, or fine-tuning. The supplied technical research reaches the same conclusion: build authoritative corpus provenance, legal-aware ingestion, BM25, multilingual dense retrieval, RRF, reranking, evidence IDs, citation validation, classification, jurisdiction separation, grounded generation, confidence/abstention, multilingual delivery, human escalation and evaluation before advanced features.

For this laptop, the practical architecture is **CPU-heavy retrieval + lightweight local fallback + API generation**. A 4 GB GPU should not be overloaded with embeddings, reranking and a large generator simultaneously.

End-to-end target architecture



The existing research specifies this core pipeline and explicitly treats knowledge graph and agentic orchestration as later-stage enhancements, not MVP dependencies.

2. What You Should NOT Do First

- Do not start with React UI.
- Do not start with a large local 7B/14B model on 4 GB VRAM.
- Do not scrape the whole internet or all patent databases.
- Do not make the LLM decide the legal category without a deterministic classification layer.
- Do not let the LLM invent citations.
- Do not mix India and international evidence in one unrestricted retrieval index.
- Do not claim languages or sources that have not been evaluated.
- Do not call the system legal advice; make it evidence-grounded decision support with escalation.

3. Research Foundation Before Coding

Create a research notebook/document with five separate labels so assumptions do not become fake facts:

- **OFFICIAL REQUIREMENT** — directly supported by the SIH problem statement.
- **AUTHORITATIVE DOMAIN SOURCE** — government/WIPO/regulatory source.
- **ENGINEERING RECOMMENDATION** — your chosen implementation approach.
- **ASSUMPTION** — must be benchmarked or confirmed.
- **FUTURE ENHANCEMENT** — useful, but not required for MVP.

Research questions you must answer

- What exact IP and regulatory question categories must the assistant answer?
- Which authorities are authoritative for each category?
- What makes a document current: publication date, effective date, amendment status, or replacement?
- How will a formulation be classified before retrieval?
- How will India and international evidence be kept separate?
- How will every answer claim be linked to evidence?
- When must the system abstain?
- Which languages will actually be evaluated?
- What benchmark will prove retrieval, classification, citation and safety quality?
- How will the corpus be refreshed when regulations change?

4. Authoritative Source Strategy

Source family	Role in system
India Code	Acts, sections, rules, regulations, notifications and other Indian legal material.
IP India	Patents, trademarks, designs, GI and related public search/e-services.
Ministry of Ayush	Ayush policy, programmes and official Ayurveda-related material.
CDSCO / Drugs & Cosmetics	Regulatory material relevant to traditional drugs and Drugs & Cosmetics Rules.
FSSAI	Ayurveda Aahara regulations and related food-law material.
WIPO / WIPO Lex	International IP treaties, traditional knowledge and genetic-resources material.
TKDL	Use only authorized/publicly accessible material; do not assume protected full-database access.
Bhashini	Multilingual translation/language infrastructure through an adapter; evaluate only supported languages.

Important: current official sources show that Ayurveda Aahara has its own FSSAI regulatory framework, while CDSCO separately publishes traditional-drug rules/material. The classifier must therefore distinguish food/Ayurveda-Aahara from Ayurvedic drugs instead of treating every Ayurvedic ingestible as one category.

5. Full Engineering Roadmap — Foundation to Production

Phase 0 — Problem, Scope & Acceptance Contract

Goal: Lock the problem and define what success means before implementation.

Build: Requirements matrix, supported domains, user personas, jurisdiction scope, supported languages, out-of-scope policy, human escalation rules.

Deliverables: Requirements → feature → test → demo traceability matrix.

Acceptance gate: Every mandatory PS capability has an owner, test and acceptance criterion.

Phase 1 — Domain Taxonomy & Regulatory Decision Tree

Goal: Turn the domain into a structured classification problem.

Build: Formulation categories, intended-use questions, claims, classical/proprietary/new-drug branches, food/cosmetic/phytopharma branches, IP protection categories and ABS/TK concepts.

Deliverables: Classification schema + decision tree + labeled seed examples.

Acceptance gate: A test set can be classified without an LLM deciding everything.

Phase 2 — Authority Matrix & Corpus Lock

Goal: Build the trustworthy knowledge foundation.

Build: Source registry with authority tier, jurisdiction, document type, URL, version, effective date, retrieval date, checksum and refresh status.

Deliverables: Source manifest + corpus policy + raw archive.

Acceptance gate: Every target question category has at least one authoritative evidence path.

Phase 3 — Document Ingestion & Legal Structure Extraction

Goal: Convert source documents into structured records without destroying legal hierarchy.

Build: PDF/HTML parsing, headings, sections, subsections, clauses, tables, page numbers, dates and provenance.

Deliverables: StructuredDocument representation + normalized corpus.

Acceptance gate: A source section can be traced back to the exact document and location.

Phase 4 — Legal-Aware Chunking

Goal: Create retrieval units that preserve legal meaning.

Build: Section/subsection/article-aware chunking, overlap only where needed, metadata propagation.

Deliverables: Chunk store with evidence metadata.

Acceptance gate: Retrieved chunks retain jurisdiction, section, version and effective-date context.

Phase 5 — BM25 Baseline

Goal: Establish a measurable lexical retrieval baseline.

Build: BM25 index and query API.

Deliverables: Baseline Recall@K, MRR and nDCG results.

Acceptance gate: Every retrieval improvement is measured against this baseline.

Phase 6 — Multilingual Dense Retrieval

Goal: Add semantic retrieval for paraphrased and multilingual questions.

Build: BGE-M3 embeddings on CPU, FAISS index, embedding cache.

Deliverables: Dense retriever + benchmark comparison.

Acceptance gate: Dense retrieval adds measurable recall without exhausting the laptop.

Phase 7 — Hybrid Fusion + Reranking

Goal: Combine exact legal terminology with semantic matching and improve top results.

Build: BM25 + dense top-K → RRF → cross-encoder reranker.

Deliverables: Hybrid retrieval service + reranker benchmark.

Acceptance gate: Reranking measurably improves Precision@K / nDCG.

Phase 8 — Evidence Object & Citation Architecture

Goal: Make evidence a first-class backend object rather than text pasted into prompts.

Build: Evidence IDs, source metadata, section/page, version, effective date, authority tier and URL.

Deliverables: Evidence store + evidence pack builder.

Acceptance gate: Every generated claim can point to a concrete evidence ID.

Phase 9 — Citation Validation

Goal: Prevent invented or mismatched legal citations.

Build: Claim extraction, evidence-ID validation, source/version/jurisdiction checks and support checks.

Deliverables: Citation validator + rejection tests.

Acceptance gate: Fake evidence IDs and unsupported claims are rejected.

Phase 10 — Grounded Generation

Goal: Generate answers only from validated evidence.

Build: Evidence-only prompting, concise answer structure, uncertainty wording and disclaimer.

Deliverables: LLM service with local fallback.

Acceptance gate: Unsupported claims are detected/rejected; grounded answers pass citation tests.

Phase 11 — Formulation Classification Engine

Goal: Route regulatory questions through a deterministic classification flow.

Build: Rule engine + minimal clarifying questions + structured classification result.

Deliverables: Classification API + labeled test set.

Acceptance gate: Classification meets agreed F1/accuracy threshold and ambiguous cases escalate.

Phase 12 — Jurisdiction Firewall

Goal: Prevent India and international evidence contamination.

Build: Separate indices/collections, typed retrieval request, jurisdiction metadata and final validation.

Deliverables: Jurisdiction-aware retrieval service.

Acceptance gate: Cross-jurisdiction leakage test returns zero leakage.

Phase 13 — Confidence, Safety & Abstention

Goal: Make the system safe when evidence is weak or conflicting.

Build: Evidence-derived confidence using retrieval, authority, coverage, classification and version signals; abstention thresholds.

Deliverables: Confidence engine + safety policy.

Acceptance gate: Low-confidence/high-stakes cases abstain or escalate instead of guessing.

Phase 14 — Multilingual Delivery

Goal: Serve evaluated languages while preserving evidence identity.

Build: Language detection, canonicalization, Bhashini adapter, answer translation and citation-preservation tests.

Deliverables: Multilingual API.

Acceptance gate: Meaning and evidence references survive translation; only evaluated languages are advertised.

Phase 15 — Human-in-the-Loop

Goal: Give users a clear path when AI cannot safely conclude.

Build: Escalation reason, evidence pack, confidence, unresolved questions and facilitator handoff.

Deliverables: HITL workflow.

Acceptance gate: Ambiguous/out-of-scope/high-stakes cases reliably escalate.

Phase 16 — Evaluation & Red-Team Benchmark

Goal: Prove the system works instead of relying on demo impressions.

Build: Retrieval, classification, generation, citation, safety, multilingual and adversarial benchmark sets.

Deliverables: Evaluation harness + reports.

Acceptance gate: Regression suite runs automatically and results are reproducible.

Phase 17 — Backend Productization

Goal: Turn research modules into a maintainable service.

Build: FastAPI, service interfaces, configuration, persistence, audit logs, error handling and health endpoints.

Deliverables: Production-shaped backend.

Acceptance gate: Core workflows run through stable APIs without notebook/manual steps.

Phase 18 — Frontend

Goal: Expose the trustworthy artifacts clearly.

Build: React/Vite interface for chat, classification, jurisdiction, evidence, citations, confidence and escalation.

Deliverables: Usable UI.

Acceptance gate: All core workflows can be completed through UI only.

Phase 19 — Security & Adversarial Hardening

Goal: Defend against prompt injection, poisoned sources, citation manipulation and unsafe inputs.

Build: Input validation, document-as-data policy, path/file limits, prompt-injection tests, access controls, audit logs.

Deliverables: Security test suite + threat model.

Acceptance gate: Known attack cases fail safely.

Phase 20 — Deployment Engineering

Goal: Make the system reproducible across laptop, SIH and larger environments.

Build: Docker/Compose profiles, environment configuration, database migration, startup checks, model provisioning and offline/local fallback.

Deliverables: Deployment bundles for dev/SIH/enterprise tiers.

Acceptance gate: Fresh environment can be provisioned and started without source edits.

Phase 21 — Observability, Backup & Corpus Refresh

Goal: Operate the system after deployment.

Build: Metrics, logs, latency tracking, retrieval diagnostics, citation rejection rate, abstention rate, database backups, restore tests and corpus refresh pipeline.

Deliverables: Operations runbook + monitoring + backup/restore procedure.

Acceptance gate: A corpus update can be validated, re-indexed, evaluated and released safely.

Phase 22 — CI/CD & Production Release

Goal: Move from project demo to controlled releases.

Build: Automated tests → evaluation gates → container build → staging smoke tests → release artifact → production deployment.

Deliverables: Versioned release + rollback procedure.

Acceptance gate: A release is reproducible, tested and rollback-capable.

Phase 23 — Production Validation

Goal: Prove production readiness.

Build: Load tests, failure recovery, multilingual regression, citation tests, jurisdiction isolation, backup/restore and deployment rehearsal.

Deliverables: Production readiness report.

Acceptance gate: All critical gates pass; unsupported capabilities remain explicitly out of scope.

6. RTX 3050 / 16 GB / 4 GB VRAM Engineering Plan

Your laptop is sufficient for development and a strong SIH prototype if you stop trying to make it behave like an enterprise GPU server.

Component	Run locally?	Recommendation
FastAPI + React	Yes	Always local.
SQLite	Yes	Development/default; keep repository abstraction so PostgreSQL can replace it.
FAISS	Yes	CPU; appropriate for a curated MVP corpus.
BM25	Yes	CPU.
BGE-M3 embeddings	Yes	CPU; batch and cache embeddings.
Cross-encoder reranker	Yes	CPU; benchmark latency and cache results where useful.
OCR/document parsing	Yes	CPU-first; process documents offline in batches.
Qwen2.5-3B 4-bit	Yes, cautiously	Local fallback; load only when needed. Do not keep multiple large models resident.
Gemini/API generation	Optional	Use for quality if external API is acceptable; keep local fallback for resilience.
Large 7B/14B local model	No as primary	Not a sensible 4 GB VRAM dependency.
Knowledge graph / multi-agent swarm	Later	Do not spend laptop resources on this during MVP.

Practical GPU rule: **one major local model at a time**. Keep embeddings and reranking on CPU so the 4 GB GPU remains available for local generation when you need offline fallback.

7. Recommended Software Stack

Layer	Choice	Reason
Frontend	React + Vite	Lightweight, production-capable UI.
Backend	FastAPI	Simple Python API and strong fit for ML services.
Retrieval	BM25 + FAISS	Exact legal terms + semantic search.
Embeddings	BGE-M3 CPU	Multilingual semantic retrieval.
Reranker	bge-reranker-v2-m3 CPU	Improve top-K precision.
Generation	Gemini API + Qwen2.5-3B Q4 fallback	Quality first with local resilience.
Database	SQLite → PostgreSQL	Simple start, credible production path.
Translation	Bhashini adapter	Keep translation provider replaceable.
Testing	pytest + integration/adversarial tests	Automated correctness gates.
Packaging	Docker / Compose	Reproducible deployment.
Evaluation	Custom benchmark harness	Retrieval + classification + citation + safety metrics.

8. Repository Structure to Start With

```
ip-sakti-sahayak/  
  backend/  
    api/  
    classification/  
    jurisdiction/  
    rag/  
    retrieval/  
    embeddings/  
    reranking/  
    fusion/  
    evidence/  
    citations/  
    generation/  
    confidence/  
    safety/  
    translation/  
    audit/  
  data/  
    raw/  
    normalized/  
    chunks/  
    manifest/  
    indexes/  
    india/  
    international/  
  evaluation/  
    dataset/  
    retrieval/  
    classification/  
    citations/  
    safety/  
    multilingual/  
  frontend/  
  tests/  
    unit/  
    integration/  
    adversarial/  
    regression/  
  config/  
    corpus.yaml  
    models.yaml  
    thresholds.yaml  
    jurisdictions.yaml  
  docs/  
    architecture/  
    corpus/  
    evaluation/  
    security/  
  scripts/  
    ingest.py  
    chunk.py  
    embed.py  
    build_index.py  
    evaluate.py  
  docker-compose.yml  
  requirements.txt  
  README.md
```

9. The First 7 Days — What You Actually Do

Day	Focus	Concrete output
Day 1	Freeze scope	Create requirements matrix, taxonomy draft, jurisdiction policy and supported-language target. No UI.
Day 2	Build source authority matrix	Identify authoritative sources and record URLs, authority, jurisdiction, document type, version/effective-date strategy.

Day 3	Collect a small India corpus	Start with a deliberately small, high-quality set of documents. Save raw files and manifest.
Day 4	Implement ingestion	Parse PDF/HTML, preserve headings/sections/pages and produce normalized structured documents.
Day 5	Implement legal-aware chunks	Create chunks with section metadata, source metadata, jurisdiction and version.
Day 6	Build BM25 baseline	Create retrieval API and benchmark 30–50 manually curated queries.
Day 7	Measure and decide	Calculate Recall@5/10, MRR and inspect failures. Only then add dense retrieval.

If Day 7 retrieval quality is bad, **do not add an LLM to hide the problem**. Fix the corpus/chunking/retrieval first.

10. First Milestone

Your first technical milestone should be a tiny but trustworthy end-to-end vertical slice:

5–10 authoritative Indian documents
↓
Versioned ingestion
↓
Legal-aware chunks
↓
BM25
↓
Top-5 evidence
↓
Simple answer generator
↓
Evidence IDs
↓
Citation validation
↓
Answer OR abstain

Do not call this an MVP chatbot yet. Call it **Retrieval + Evidence Validation Prototype v0**. Once this is reliable, add dense retrieval and reranking.

- Enterprise authentication/RBAC and stronger multi-user isolation.
- Advanced observability and analytics.

The research explicitly recommends treating agentic orchestration and knowledge-graph capabilities as future evolution rather than MVP dependencies.

14. Research Sources & Provenance

Existing project research

PS_26045_IP_SAKTI_Sahayak_Deep_Technical_Research.pdf — supplied project research artifact used as the primary engineering baseline.

Problem statement

PS_26045_IP_SAKTI_Sahayak.pdf — supplied SIH problem statement artifact.

India Code

<https://www.indiacode.nic.in/> — official repository/search interface for Indian Acts, sections, subordinate legislation, notifications and related material.

IP India

<https://ipindia.gov.in/> — official Intellectual Property India portal and public search/e-services.

Ministry of Ayush

<https://ayush.gov.in/> — official Ministry of Ayush portal.

CDSCO Traditional Drugs

https://www.cdsc.gov.in/opencms/opencms/en/Traditional_Drugs/ — official traditional-drug regulatory information.

CDSCO Drugs Rules

<https://www.cdsc.gov.in/opencms/opencms/en/Acts-and-rules/Drugs-Rules/> — official rules/notifications repository.

FSSAI Ayurveda Aahara

https://fssai.gov.in/upload/notifications/2022/05/62789a20b54bdGazette_Notification_Ayurveda_Aahara_09_05_2022.pdf — official Ayurveda Aahara Regulations, 2022 gazette document.

FSSAI advisories/orders

<https://www.fssai.gov.in/food-law/advisories> — official food-law advisories/orders including Ayurveda Aahara updates.

WIPO GRATK

<https://www.wipo.int/en/web/traditional-knowledge/wipo-treaty-on-ip-gr-and-associated-tk> — WIPO resource center for the 2024 treaty on IP, genetic resources and associated traditional knowledge.

WIPO Lex

<https://www.wipo.int/wipolex/en/treaties/textdetails/19849> — official WIPO treaty text record.

Bhashini AI

<https://app.bhashini.ai/translate> — translation/API interface; verify exact service terms and supported languages before production use.

15. Final Build Order

Do this in exactly this order unless an experiment proves a change is necessary:

1. Scope + requirements
2. Domain taxonomy + classification tree
3. Authority matrix
4. Small authoritative corpus
5. Ingestion + provenance
6. Legal-aware chunking
7. BM25 baseline
8. Retrieval benchmark
9. BGE-M3 + FAISS
10. RRF + reranker
11. Evidence IDs
12. Citation validator
13. Grounded LLM
14. Classification engine
15. Jurisdiction firewall
16. Confidence + abstention
17. Multilingual
18. Human escalation
19. Evaluation + red-team
20. FastAPI productization
21. React UI
22. Security hardening
23. Docker deployment
24. Monitoring + backups
25. Corpus refresh
26. CI/CD
27. Production validation
28. Release

Bottom line

Your RTX 3050 laptop is **good enough to build the full research prototype and SIH-ready system**. The mistake would be spending your limited VRAM on a large local model. Put the engineering effort into the real moat: authoritative corpus provenance, legal-aware retrieval, classification, jurisdiction isolation, evidence binding, citation validation, safe abstention and measurable evaluation.

PS 26045 — IP-SAKTI Sahayak

28-Part Research Compendium

Multilingual RAG-based IP / Regulatory Assistant for Ayurveda

Research → Architecture → Audit → Production Blueprint

Development hardware: NVIDIA RTX 3050 · 4 GB VRAM · 16 GB RAM · 512 GB SSD

Consolidated from the validated project research artifacts and final architecture decisions.

Source discipline: [OFFICIAL PS] · [OFFICIAL SOURCE] · [EXTERNAL RESEARCH] · [ENGINEERING RECOMMENDATION] · [OUR ENHANCEMENT] · [ASSUMPTION] · [DEFERRED]

Compilation / Provenance Note

This PDF is a consolidation of the validated PS 26045 research material currently available in the project workspace. The primary uploaded research artifact is the 14-page *PS_26045_IP_SAKTI_Deep_Technical_Research_Roadmap_RTX3050.pdf*, supplemented here by the final architecture and implementation decisions established in the research sequence. The source artifact itself distinguishes official requirements from engineering recommendations and future enhancements; this compendium preserves that distinction.

Important: historical chat turns are not treated as independently authoritative when a validated project artifact exists. Where a decision was later corrected, the later validated decision is presented as the current engineering position.

28-Part Research Index

Part	Research Area
Part 01	Problem Understanding
Part 02	Exact PS Requirements
Part 03	Core System Decomposition
Part 04	Source / Data Authority Research
Part 05	Dataset / Corpus Strategy
Part 06	RAG Research
Part 07	LLM Research
Part 08	Formulation Classification
Part 09	Jurisdiction Architecture
Part 10	Citation / Hallucination Control
Part 11	Multilingual AI
Part 12	Agentic AI
Part 13	Knowledge Graph
Part 14	Security & Privacy
Part 15	Complete Tech Stack
Part 16	Laptop Feasibility Audit
Part 17	Model Routing / Resource Management
Part 18	Evaluation
Part 19	Baseline vs Advanced Versions
Part 20	Technical Novelty
Part 21	Production Deployment Architecture
Part 22	Production Observability & Maintenance
Part 23	Hardware, Performance & Cost
Part 24	Existing Solutions & Competitor Analysis
Part 25	Technical Novelty & Contribution
Part 26	Complete Production Implementation Roadmap
Part 27	Final Architecture Audit
Part 28	Final Master Blueprint / Architecture Decision Lock

Validated Research Artifact — Full Extracted Content

The following section preserves the textual content extracted from the uploaded PS 26045 technical roadmap artifact. Page boundaries are retained for traceability.

Source Artifact — Page 1

PS 26045 — IP-SAKTI Sahayak | Technical Research & Roadmap Page 1 PS 26045 — IP-SAKTI Sahayak Deep Technical Research, Engineering Roadmap & Production Plan Multilingual RAG-based IP/Regulatory Assistant for Ayurveda Target development hardware Component Constraint Engineering implication GPU NVIDIA RTX 3050 / 4 GB VRAM Do not make a large local LLM the primary dependency; reserve GPU for one local generation model when needed. RAM 16 GB Keep the service stack lightweight; run embeddings/reranking on CPU where practical. Storage 512 GB SSD Use a curated authoritative corpus and avoid mass scraping; keep raw/normalized/indexed data separated. Target SIH → credible production path Design interfaces so storage, model and deployment tiers can change without rewriting the core pipeline. Core engineering principle: The project is not a generic Ayurveda chatbot. It is a classification-first, jurisdiction-aware, evidence-grounded regulatory decision-support system. The LLM is only one component. Prepared as a build-before-code research blueprint. Source-derived requirements and architecture are separated from engineering recommendations and future enhancements.

1. Executive Engineering Verdict

The correct starting point is corpus and retrieval validation, not UI, agents, voice, a knowledge graph, or fine-tuning. The supplied technical research reaches the same conclusion: build authoritative corpus provenance, legal-aware ingestion, BM25, multilingual dense retrieval, RRF, reranking, evidence IDs, citation validation, classification, jurisdiction separation, grounded generation, confidence/abstention, multilingual delivery, human escalation and evaluation before advanced features. For this laptop, the practical architecture is CPU-heavy retrieval + lightweight local fallback + API generation. A 4 GB GPU should not be overloaded with embeddings, reranking and a large generator simultaneously. End-to-end target architecture User ↓ Language Detection / Canonicalization ↓ Formulation Classification ↓ Jurisdiction Firewall ↓ India / International Retrieval ■■■ BM25 ■■■ BGE-M3 + FAISS ↓ RRF Fusion ↓ Cross-Encoder Reranker ↓ Evidence Pack ↓ Gemini / Local Qwen ↓ Claim ↔ Evidence Binding ↓ Citation Validation ↓ Confidence + Safety ■■■ Answer ■■■ Abstain / Human Escalation The existing research specifies this core pipeline and explicitly treats knowledge graph and agentic orchestration as later-stage enhancements, not MVP dependencies.

2. What You Should NOT Do First

■ Do not start with React UI. ■ Do not start with a large local 7B/14B model on 4 GB VRAM. ■ Do not scrape the whole internet or all patent databases. ■ Do not make the LLM decide the legal category without a deterministic classification layer. ■ Do not let the LLM invent citations. ■ Do not mix India and international evidence in one unrestricted retrieval index. ■ Do not claim languages or sources that have not been evaluated. ■ Do not call the system legal advice; make it evidence-grounded decision support with escalation.

3. Research Foundation Before Coding

Create a research notebook/document with five separate labels so assumptions do not become fake facts: ■ OFFICIAL REQUIREMENT — directly supported by the SIH problem statement. ■ AUTHORITATIVE DOMAIN SOURCE — government/WIPO/regulatory source. ■ ENGINEERING RECOMMENDATION — your chosen implementation approach. ■ ASSUMPTION — must be benchmarked or confirmed. ■ FUTURE ENHANCEMENT — useful, but not required for MVP. Research questions you must answer ■ What exact IP and regulatory question categories must the assistant answer? ■ Which authorities are authoritative for each category? ■ What makes a document current: publication date, effective date, amendment status, or replacement? ■ How will a formulation be classified before retrieval? ■ How will India and international evidence be kept separate? ■ How will every answer claim be linked to evidence? ■ When must the system abstain? ■ Which languages will actually be evaluated? ■ What benchmark will prove retrieval, classification, citation and safety quality? ■ How will the corpus be refreshed when regulations change?

4. Authoritative Source Strategy

Source family Role in system India Code Acts, sections, rules, regulations, notifications and other Indian legal material. IP India Patents, trademarks, designs, GI and related public search/e-services. Ministry of Ayush Ayush policy, programmes and official Ayurveda-related material. CDSCO / Drugs & Cosmetics Regulatory material relevant to traditional drugs and Drugs & Cosmetics Rules. FSSAI Ayurveda Aahara regulations and related food-law material. WIPO / WIPO Lex International IP treaties, traditional knowledge and genetic-resources material. TKDL Use only authorized/publicly accessible material; do not assume protected full-database access. Bhashini Multilingual translation/language infrastructure through an adapter; evaluate only supported languages. Important: current official sources show that Ayurveda Aahara has its own FSSAI regulatory framework, while CDSCO separately publishes traditional-drug rules/material. The classifier must therefore distinguish food/Ayurveda-Aahara from Ayurvedic drugs instead of treating every Ayurvedic ingestible as one category.

5. Full Engineering Roadmap — Foundation to Production

Phase 0 — Problem, Scope & Acceptance Contract

Goal: Lock the problem and define what success means before implementation. Build: Requirements matrix, supported domains, user personas, jurisdiction scope, supported languages, out-of-scope policy, human escalation rules. Deliverables: Requirements → feature → test → demo traceability matrix. Acceptance gate: Every mandatory PS capability has an owner, test and acceptance criterion.

Phase 1 — Domain Taxonomy & Regulatory Decision Tree

Goal: Turn the domain into a structured classification problem. Build: Formulation categories, intended-use questions, claims, classical/proprietary/new-drug branches, food/cosmetic/phytopharma branches, IP protection categories and ABS/TK concepts. Deliverables: Classification schema + decision tree + labeled seed examples. Acceptance gate: A test set can be classified without an LLM deciding everything.

Phase 2 — Authority Matrix & Corpus Lock

Goal: Build the trustworthy knowledge foundation. Build: Source registry with authority tier, jurisdiction, document type, URL, version, effective date, retrieval date, checksum and refresh status. Deliverables: Source manifest + corpus policy + raw archive. Acceptance gate: Every target question category has at least one authoritative evidence path.

Phase 3 — Document Ingestion & Legal Structure Extraction

Goal: Convert source documents into structured records without destroying legal hierarchy. Build: PDF/HTML parsing, headings, sections, subsections, clauses, tables, page numbers, dates and provenance. Deliverables: StructuredDocument representation + normalized corpus. Acceptance gate: A source section can be traced back to the exact document and location.

Phase 4 — Legal-Aware Chunking

Goal: Create retrieval units that preserve legal meaning. Build: Section/subsection/article-aware chunking, overlap only where needed, metadata propagation. Deliverables: Chunk store with evidence metadata. Acceptance gate: Retrieved chunks retain jurisdiction, section, version and effective-date context.

Phase 5 — BM25 Baseline

Goal: Establish a measurable lexical retrieval baseline. Build: BM25 index and query API. Deliverables: Baseline Recall@K, MRR and nDCG results. Acceptance gate: Every retrieval improvement is measured against this baseline.

Source Artifact — Page 5

PS 26045 — IP-SAKTI Sahayak | Technical Research & Roadmap Page 5

Phase 6 — Multilingual Dense Retrieval

Goal: Add semantic retrieval for paraphrased and multilingual questions. Build: BGE-M3 embeddings on CPU, FAISS index, embedding cache. Deliverables: Dense retriever + benchmark comparison. Acceptance gate: Dense retrieval adds measurable recall without exhausting the laptop.

Phase 7 — Hybrid Fusion + Reranking

Goal: Combine exact legal terminology with semantic matching and improve top results. Build: BM25 + dense top-K → RRF → cross-encoder reranker. Deliverables: Hybrid retrieval service + reranker benchmark. Acceptance gate: Reranking measurably improves Precision@K / nDCG.

Phase 8 — Evidence Object & Citation Architecture

Goal: Make evidence a first-class backend object rather than text pasted into prompts. Build: Evidence IDs, source metadata, section/page, version, effective date, authority tier and URL. Deliverables: Evidence store + evidence pack builder. Acceptance gate: Every generated claim can point to a concrete evidence ID.

Phase 9 — Citation Validation

Goal: Prevent invented or mismatched legal citations. Build: Claim extraction, evidence-ID validation, source/version/jurisdiction checks and support checks. Deliverables: Citation validator + rejection tests. Acceptance gate: Fake evidence IDs and unsupported claims are rejected.

Phase 10 — Grounded Generation

Goal: Generate answers only from validated evidence. Build: Evidence-only prompting, concise answer structure, uncertainty wording and disclaimer. Deliverables: LLM service with local fallback. Acceptance gate: Unsupported claims are detected/rejected; grounded answers pass citation tests.

Phase 11 — Formulation Classification Engine

Goal: Route regulatory questions through a deterministic classification flow. Build: Rule engine + minimal clarifying questions + structured classification result. Deliverables: Classification API + labeled test set. Acceptance gate: Classification meets agreed F1/accuracy threshold and ambiguous cases escalate.

Source Artifact — Page 6

PS 26045 — IP-SAKTI Sahayak | Technical Research & Roadmap Page 6

Phase 12 — Jurisdiction Firewall

Goal: Prevent India and international evidence contamination. Build: Separate indices/collections, typed retrieval request, jurisdiction metadata and final validation. Deliverables: Jurisdiction-aware retrieval service. Acceptance gate: Cross-jurisdiction leakage test returns zero leakage.

Phase 13 — Confidence, Safety & Abstention

Goal: Make the system safe when evidence is weak or conflicting. Build: Evidence-derived confidence using retrieval, authority, coverage, classification and version signals; abstention thresholds. Deliverables: Confidence engine + safety policy. Acceptance gate: Low-confidence/high-stakes cases abstain or escalate instead of guessing.

Phase 14 — Multilingual Delivery

Goal: Serve evaluated languages while preserving evidence identity. Build: Language detection, canonicalization, Bhashini adapter, answer translation and citation-preservation tests. Deliverables: Multilingual API. Acceptance gate: Meaning and evidence references survive translation; only evaluated languages are advertised.

Phase 15 — Human-in-the-Loop

Goal: Give users a clear path when AI cannot safely conclude. Build: Escalation reason, evidence pack, confidence, unresolved questions and facilitator handoff. Deliverables: HITL workflow. Acceptance gate: Ambiguous/out-of-scope/high-stakes cases reliably escalate.

Phase 16 — Evaluation & Red-Team Benchmark

Goal: Prove the system works instead of relying on demo impressions. Build: Retrieval, classification, generation, citation, safety, multilingual and adversarial benchmark sets. Deliverables: Evaluation harness + reports. Acceptance gate: Regression suite runs automatically and results are reproducible.

Phase 17 — Backend Productization

Goal: Turn research modules into a maintainable service. Build: FastAPI, service interfaces, configuration, persistence, audit logs, error handling and health endpoints. Deliverables: Production-shaped backend. Acceptance gate: Core workflows run through stable APIs without notebook/manual steps.

Source Artifact — Page 7

PS 26045 — IP-SAKTI Sahayak | Technical Research & Roadmap Page 7

Phase 18 — Frontend

Goal: Expose the trustworthy artifacts clearly. Build: React/Vite interface for chat, classification, jurisdiction, evidence, citations, confidence and escalation. Deliverables: Usable UI. Acceptance gate: All core workflows can be completed through UI only.

Phase 19 — Security & Adversarial Hardening

Goal: Defend against prompt injection, poisoned sources, citation manipulation and unsafe inputs. Build: Input validation, document-as-data policy, path/file limits, prompt-injection tests, access controls, audit logs. Deliverables: Security test suite + threat model. Acceptance gate: Known attack cases fail safely.

Phase 20 — Deployment Engineering

Goal: Make the system reproducible across laptop, SIH and larger environments. Build: Docker/Compose profiles, environment configuration, database migration, startup checks, model provisioning and offline/local fallback. Deliverables: Deployment bundles for dev/SIH/enterprise tiers. Acceptance gate: Fresh environment can be provisioned and started without source edits.

Phase 21 — Observability, Backup & Corpus Refresh

Goal: Operate the system after deployment. Build: Metrics, logs, latency tracking, retrieval diagnostics, citation rejection rate, abstention rate, database backups, restore tests and corpus refresh pipeline. Deliverables: Operations runbook + monitoring + backup/restore procedure. Acceptance gate: A corpus update can be validated, re-indexed, evaluated and released safely.

Phase 22 — CI/CD & Production Release

Goal: Move from project demo to controlled releases. Build: Automated tests → evaluation gates → container build → staging smoke tests → release artifact → production deployment. Deliverables: Versioned release + rollback procedure. Acceptance gate: A release is reproducible, tested and rollback-capable.

Phase 23 — Production Validation

Goal: Prove production readiness. Build: Load tests, failure recovery, multilingual regression, citation tests, jurisdiction isolation, backup/restore and deployment rehearsal. Deliverables: Production readiness report. Acceptance gate: All critical gates pass; unsupported capabilities remain explicitly out of scope.

6. RTX 3050 / 16 GB / 4 GB VRAM Engineering Plan

Your laptop is sufficient for development and a strong SIH prototype if you stop trying to make it behave like an enterprise GPU server. Component Run locally? Recommendation FastAPI + React Yes Always local. SQLite Yes Development/default; keep repository abstraction so PostgreSQL can replace it. FAISS Yes CPU; appropriate for a curated MVP corpus. BM25 Yes CPU. BGE-M3 embeddings Yes CPU; batch and cache embeddings. Cross-encoder reranker Yes CPU; benchmark latency and cache results where useful. OCR/document parsing Yes CPU-first; process documents offline in batches. Qwen2.5-3B 4-bit Yes, cautiously Local fallback; load only when needed. Do not keep multiple large models resident. Gemini/API generation Optional Use for quality if external API is acceptable; keep local fallback for resilience. Large 7B/14B local model No as primary Not a sensible 4 GB VRAM dependency. Knowledge graph / multi-agent swarm Later Do not spend laptop resources on this during MVP. Practical GPU rule: one major local model at a time. Keep embeddings and reranking on CPU so the 4 GB GPU remains available for local generation when you need offline fallback.

7. Recommended Software Stack

Layer Choice Reason Frontend React + Vite Lightweight, production-capable UI. Backend FastAPI Simple Python API and strong fit for ML services. Retrieval BM25 + FAISS Exact legal terms + semantic search. Embeddings BGE-M3 CPU Multilingual semantic retrieval. Reranker bge-reranker-v2-m3 CPU Improve top-K precision. Generation Gemini API + Qwen2.5-3B Q4 fallback Quality first with local resilience. Database SQLite → PostgreSQL Simple start, credible production path. Translation Bhashini adapter Keep translation provider replaceable. Testing pytest + integration/adversarial tests Automated correctness gates. Packaging Docker / Compose Reproducible deployment. Evaluation Custom benchmark harness Retrieval + classification + citation + safety metrics.

8. Repository Structure to Start With

ip-sakti-sahayak/ ■■■■ backend/ ■■■■ api/ ■■■■ classification/ ■■■■ jurisdiction/ ■■■■ rag/ ■■■■ retrieval/ ■■■■
■■■■ embeddings/ ■■■■ reranking/ ■■■■ fusion/ ■■■■ evidence/ ■■■■ citations/ ■■■■ generation/ ■■■■
confidence/ ■■■■ safety/ ■■■■ translation/ ■■■■ audit/ ■■■■ data/ ■■■■ raw/ ■■■■ normalized/ ■■■■ chunks/ ■■■■
■■■■ manifest/ ■■■■ indexes/ ■■■■ india/ ■■■■ international/ ■■■■ evaluation/ ■■■■ dataset/ ■■■■ retrieval/ ■■■■
classification/ ■■■■ citations/ ■■■■ safety/ ■■■■ multilingual/ ■■■■ frontend/ ■■■■ tests/ ■■■■ unit/ ■■■■ integration/
■■■■ adversarial/ ■■■■ regression/ ■■■■ config/ ■■■■ corpus.yaml ■■■■ models.yaml ■■■■ thresholds.yaml ■■■■
jurisdictions.yaml ■■■■ docs/ ■■■■ architecture/ ■■■■ corpus/ ■■■■ evaluation/ ■■■■ security/ ■■■■ scripts/ ■■■■
ingest.py ■■■■ chunk.py ■■■■ embed.py ■■■■ build_index.py ■■■■ evaluate.py ■■■■ docker-compose.yml ■■■■
requirements.txt ■■■■ README.md

9. The First 7 Days — What You Actually Do

Day Focus Concrete output Day 1 Freeze scope Create requirements matrix, taxonomy draft, jurisdiction policy and supported-language target. No UI. Day 2 Build source authority matrix Identify authoritative sources and record URLs, authority, jurisdiction, document type, version/effective-date strategy.

Source Artifact — Page 10

PS 26045 — IP-SAKTI Sahayak | Technical Research & Roadmap Page 10 Day 3 Collect a small India corpus Start with a deliberately small, high-quality set of documents. Save raw files and manifest. Day 4 Implement ingestion Parse PDF/HTML, preserve headings/sections/pages and produce normalized structured documents. Day 5 Implement legal-aware chunks Create chunks with section metadata, source metadata, jurisdiction and version. Day 6 Build BM25 baseline Create retrieval API and benchmark 30–50 manually curated queries. Day 7 Measure and decide Calculate Recall@5/10, MRR and inspect failures. Only then add dense retrieval. If Day 7 retrieval quality is bad, do not add an LLM to hide the problem. Fix the corpus/chunking/retrieval first.

10. First Milestone

Your first technical milestone should be a tiny but trustworthy end-to-end vertical slice: 5–10 authoritative Indian documents ↓ Versioned ingestion ↓ Legal-aware chunks ↓ BM25 ↓ Top-5 evidence ↓ Simple answer generator ↓ Evidence IDs ↓ Citation validation ↓ Answer OR abstain Do not call this an MVP chatbot yet. Call it Retrieval + Evidence Validation Prototype v0. Once this is reliable, add dense retrieval and reranking.

Source Artifact — Page 11

PS 26045 — IP-SAKTI Sahayak | Technical Research & Roadmap Page 11

11. Evaluation Plan

Area Metric Initial target philosophy Area Metric Initial target philosophy Retrieval Recall@5 / Recall@10 / MRR / nDCG
Compare every retrieval version against BM25 baseline. Classification Accuracy / macro F1 / confusion matrix Set threshold
before SIH; ambiguous cases should escalate. Generation Correctness / faithfulness / unsupported claim rate Evidence support
matters more than fluent prose. Citation Citation precision / recall / invalid citation rate Invalid citations should approach zero on
benchmark. Safety Hallucination / safe abstention / false abstention Prefer safe abstention over confident unsupported claims.
Multilingual Retrieval quality / correctness / citation preservation Evaluate each language independently. Latency p50 / p95
Track retrieval and generation separately. System Error rate / health / recovery A demo that cannot recover is not
production-ready.

12. Production Architecture

Reverse Proxy ↓ React UI ↓ FastAPI ↓ ██ ■ Regulatory Core ■ ■ language ■ ■ classification
■ ■ jurisdiction ■ ■ retrieval ■ ■ evidence ■ ■ citation ■ ■ confidence ■ ■ safety ■ ██ ↓
PostgreSQL + Vector Store ↓ Gemini / Local Model ↓ Audit + Observability Background: Corpus refresh → validation → re-index
→ evaluation → release Production should not simply mean “Dockerized.” It means reproducible deployment, controlled corpus
updates, evaluation gates, monitoring, backup/restore, security testing and rollback.

13. Future Evolution — Only After SIH-Ready Core

■ Single-agent tool orchestration for registry/TKDL/ABS research workflows. ■ Knowledge graph for entities such as formulations, ingredients, laws, authorities and jurisdictions. ■ Graph + RAG multi-hop evidence retrieval. ■ More live registry/search connectors where access and terms permit. ■ Expanded language coverage after native-speaker evaluation.

Source Artifact — Page 12

PS 26045 — IP-SAKTI Sahayak | Technical Research & Roadmap Page 12 ■ Enterprise authentication/RBAC and stronger multi-user isolation. ■ Advanced observability and analytics. The research explicitly recommends treating agentic orchestration and knowledge-graph capabilities as future evolution rather than MVP dependencies.

14. Research Sources & Provenance

Existing project research PS_26045_IP_SAKTI_Sahayak_Deep_Technical_Research.pdf — supplied project research artifact used as the primary engineering baseline. Problem statement PS_26045_IP_SAKTI_Sahayak.pdf — supplied SIH problem statement artifact. India Code <https://www.indiacode.nic.in/> — official repository/search interface for Indian Acts, sections, subordinate legislation, notifications and related material. IP India <https://ipindia.gov.in/> — official Intellectual Property India portal and public search/e-services. Ministry of Ayush <https://ayush.gov.in/> — official Ministry of Ayush portal. CDSCO Traditional Drugs https://www.cdsc.gov.in/opencms/opencms/en/Traditional_Drugs/ — official traditional-drug regulatory information. CDSCO Drugs Rules <https://www.cdsc.gov.in/opencms/opencms/en/Acts-and-rules/Drugs-Rules/> — official rules/notifications repository. FSSAI Ayurveda Aahara https://fssai.gov.in/upload/notifications/2022/05/62789a20b54bdGazette_Notification_Ayurveda_Aahara_09_05_2022.pdf — official Ayurveda Aahara Regulations, 2022 gazette document. FSSAI advisories/orders <https://www.fssai.gov.in/food-law/advisories> — official food-law advisories/orders including Ayurveda Aahara updates. WIPO GRATK <https://www.wipo.int/en/web/traditional-knowledge/wipo-treaty-on-ip-gr-and-associated-tk> — WIPO resource center for the 2024 treaty on IP, genetic resources and associated traditional knowledge. WIPO Lex <https://www.wipo.int/wipolex/en/treaties/textdetails/19849> — official WIPO treaty text record. Bhashini AI <https://app.bhashini.ai/translate> — translation/API interface; verify exact service terms and supported languages before production use.

15. Final Build Order

Do this in exactly this order unless an experiment proves a change is necessary:

1. Scope + requirements
2. Domain taxonomy + classification tree
3. Authority matrix
4. Small authoritative corpus
5. Ingestion + provenance
6. Legal-aware chunking
7. BM25 baseline
8. Retrieval benchmark
9. BGE-M3 + FAISS
10. RRF + reranker
11. Evidence IDs
12. Citation validator
13. Grounded LLM
14. Classification engine
15. Jurisdiction firewall
16. Confidence + abstention
17. Multilingual

- 18. Human escalation**
- 19. Evaluation + red-team**
- 20. FastAPI productization**
- 21. React UI**
- 22. Security hardening**
- 23. Docker deployment**
- 24. Monitoring + backups**
- 25. Corpus refresh**
- 26. CI/CD**
- 27. Production validation**
- 28. Release**

Source Artifact — Page 14

PS 26045 — IP-SAKTI Sahayak | Technical Research & Roadmap Page 14 Bottom line Your RTX 3050 laptop is good enough to build the full research prototype and SIH-ready system. The mistake would be spending your limited VRAM on a large local model. Put the engineering effort into the real moat: authoritative corpus provenance, legal-aware retrieval, classification, jurisdiction isolation, evidence binding, citation validation, safe abstention and measurable evaluation.

Final Architecture Decision Lock

Final Architecture Principle

[ENGINEERING RECOMMENDATION] IP-SAKTI should be treated as a policy-governed, evidence-first regulatory intelligence system rather than a generic chatbot. The LLM is one component inside a pipeline that performs classification, jurisdiction control, retrieval, evidence binding, citation validation, confidence/safety evaluation and abstention/escalation.

Final Core Pipeline

User → Language → Classification → Jurisdiction Firewall → Hybrid Retrieval → RRF → Reranker → Evidence Pack → Generation → Claim/Evidence Binding → Citation Validation → Confidence/Safety → Answer or Escalation.

Final Core Stack

[ENGINEERING RECOMMENDATION] FastAPI backend; React/Vite frontend; Python orchestration; BM25 + multilingual dense retrieval; FAISS for laptop-oriented dense indexing; BGE-M3 CPU candidate; bge-reranker-v2-m3 CPU candidate; Gemini Flash API as primary generation candidate; Qwen2.5-3B 4-bit as local fallback candidate; Bhashini adapter for multilingual delivery; SQLite during early development with PostgreSQL as the production relational target; Docker Compose for initial deployment.

Regulatory Safety

[ENGINEERING RECOMMENDATION] Regulatory answers must be grounded in authoritative evidence, must respect jurisdiction and document version, and must not be presented as legal determinations. Missing or conflicting evidence should trigger transparent uncertainty and human escalation.

Jurisdiction Firewall

[ENGINEERING RECOMMENDATION] India and international corpora should be separated at retrieval time, not merely mentioned in the prompt. The final evidence pack must satisfy the requested jurisdiction.

Evidence and Citation

[ENGINEERING RECOMMENDATION] Evidence IDs are backend-owned records. The model does not invent source identifiers. Each claim should bind to actual retrieved evidence; invalid or unsupported citations are rejected.

Classification

[ENGINEERING RECOMMENDATION] Use a deterministic, explainable formulation-classification flow with structured questions. Output should be provisional classification, basis, missing information and human-review status.

Multilingual

[ENGINEERING RECOMMENDATION] Preserve the original query and canonical representation, use multilingual retrieval where validated, and translate the final answer without translating away exact section/article identifiers.

Agents and Knowledge Graph

[DEFERRED] Do not make multi-agent orchestration or a dedicated knowledge graph core dependencies. Introduce them only when benchmarked use cases demonstrate measurable value.

Hardware Lock

[ENGINEERING RECOMMENDATION] On the RTX 3050 4 GB / 16 GB RAM / 512 GB SSD development machine, avoid making a large local LLM the primary dependency. Keep embeddings/reranking on CPU where practical and avoid simultaneous residency of multiple large GPU models.

Production Readiness

[ENGINEERING RECOMMENDATION] Production means reproducible deployment, controlled corpus updates, evaluation gates, monitoring, backups/restores, security controls and rollback—not merely Dockerization.

Regulatory Freshness

[OUR ENHANCEMENT] Source change → verification → versioning → re-ingestion → re-indexing → impact analysis → evaluation → approval → production release.

Final Decision Lock

[ENGINEERING RECOMMENDATION] The project should now move from research to implementation. New technologies must earn their place through measurable evidence; they should not be added simply because they sound advanced.

Final Production Gate

- Authoritative corpus and provenance validated
- Legal-aware document structure and versioning implemented
- Retrieval benchmark completed
- Classification benchmark completed
- Jurisdiction leakage tests pass
- Citation validation tests pass
- Unsupported-claim / abstention tests pass
- Multilingual quality tested for every advertised language
- Security and prompt-injection tests pass
- Backup and restore verified
- Monitoring and health checks operational
- Corpus refresh workflow tested
- Rollback procedure tested
- Production deployment reproducible from a clean environment